Learning Objective 3 Testing Report

snnTorch software testing

Report prepared 7 November 2026 | Testing began 2 November 2026 | Windows CPU | Python 3.12.14 | PyTorch 2.15.0

Scope and result

I completed a final run of 1,134 LO3 test cases; all passed with no failures or skips. The 68 existing tests also passed. The campaign found and fixed 10 distinct defects. I collected evidence against all six requirements from the LO1 analysis and LO2 plan. Full acceptance remains open for the usability requirement because no participant sessions were run, and for one extraction case that needs a real pre-existing symbolic link on a native filesystem.

Measure	Recorded result
Final LO3 suite	1,134 passed; 0 failed; 0 skipped
Existing suite	68 passed; 0 failed; 0 skipped
Confirmed defects	10 found and fixed
Evidence and acceptance	Evidence mapped to 6/6 requirements; selected-scope pass summary 5/6
Open checks	0/5 user sessions; native existing-symbolic-link extraction case not run

Test setup

I began the run series from baseline commit eebfaa883f44c735b3e42cd881973ba42adf8fac. The LO3 tests, runner and defect fixes are preserved on the lo3-testing branch, currently at c3e286ae5e8a5e9d4b0abaa42fe436b8f31be352. Execution was limited to Windows on CPU with Python 3.12.14 and PyTorch 2.15.0. The runner records the test, coverage and smoke outputs under portfolio-testing/lo3/evidence/final. The requirement IDs below follow the LO1 and LO2 documents [1].

3 1 Testing techniques

I compared the implementation with the LO2 plan, which called for independent numerical oracles, systematic input selection, stochastic checks, state-isolation tests, acceptance testing and security tests. I used these methods where the code and test environment allowed. It also added structural tests after reviewing coverage.

Requirement	Techniques and level	What was checked
R1 Neuron dynamics	Functional unit tests and selected integration checks	Independent hand-calculated and scalar references for Leaky dynamics and selected Synaptic cases. Checked spikes, membrane or synaptic state, reset choices, thresholds, batches, dtypes and invalid inputs.

Requirement	Techniques and level	What was checked
R2 Surrogate gradients	Functional unit tests and autograd integration	Tested hard-spike forward outputs separately from surrogate backward formulas across selected gradient families, signs, parameters, dtypes and support boundaries. Checked gradients reaching inputs and trainable parameters.
R3 Spike encoders	Functional and statistical tests	Checked exact rate boundaries, latency timing and one-hot outputs. For intermediate rate probabilities, used 10,000 draws at p=0.1, 0.5 and 0.9 with three fixed seeds per probability.
R4 Model-state isolation	Model-based and integration tests	Ran 1,000 declared model and sequence combinations. Compared a reset model with a fresh model, interleaved two models, and exercised instance, class, nested and repeated reset paths.
R5 Usability	System smoke test; acceptance sessions planned	Ran the README example for 25 steps and checked an output shape of (25, 1, 10). A small encoder-to-optimizer run checked finite gradients and a weight update. No representative-user sessions were completed.
R6 Dataset and download security	Security-focused unit and integration tests	Used temporary archives and an external canary to check traversal, absolute paths and escaping links. Stubbed download failures and redirects to check HTTPS handling, cleanup and rejection of HTTP downgrades.

I selected 12 configurations from a space of 144 settings combinations. Those cases covered all 95 required feasible pairs. Boundary and invalid-input cases supplemented the pairwise set. I added 13 tests after a structural coverage review. The runner also executes the existing package tests, and the assessment script independently recomputes pair coverage from the manifest [2, 3, 5, 6].

I partly completed the plan for R5 and one R6 case. The README and training smoke tests check that the workflow runs, but they do not measure whether a beginner can complete it unaided. Windows did not permit creation of the real symbolic link needed for the remaining extraction integration case.

3 2 Evaluation criteria

I chose pass criteria to match the type of requirement. Discrete spike outputs and deterministic encoder results were compared exactly. Floating-point traces and gradients used tolerances fixed before execution: float32 rtol=1e-5 and atol=1e-6; float64 rtol=1e-10 and atol=1e-12 [5].

- For R1, spikes had to match exactly and intermediate membrane values had to match the independent recurrence within the dtype tolerance. Checking every step can expose an error that a matching final output would hide.
- For R2, the forward spike rule and backward surrogate derivative were checked as separate contracts. The spike output had to match exactly; gradients were compared with the written formula within the fixed numeric tolerance. The integration case also checked gradients for input, weights, beta and threshold.
- For R3, deterministic cases used exact expected values. Each stochastic rate count was tested against a two-sided equal-tail exact binomial interval. The family-wise error target was 0.01, adjusted across all nine counts with Bonferroni correction.

- For R4, after reset the output and state had to equal those of a fresh model on the same sequence. Resetting one model also had to leave its peer unchanged. Coverage was recorded over the four abstract two-model states and 16 state/action transitions.
- For R6, an attack fixture passed only when extraction rejected it and a canary outside the destination remained byte-for-byte unchanged. Download failures had to preserve HTTPS, avoid insecure redirects, and clean up partial files. These invariants are stronger than checking for an exception alone.
- For R5, the planned acceptance threshold remains at least four of five representative users completing the task within 30 minutes without instructor help. The automated smoke test is supporting evidence only; it cannot replace the user sessions.

The rate tests use fixed seeds and a declared sample size, which makes failures repeatable and limits false rejection under the chosen statistical model. They still sample only nine preselected runs. The mathematical references are implemented separately from production code, but their expected behaviour still depends on interpreting the same source specification.

Limits on the criteria

The source material conflicts on whether a neuron fires exactly at threshold, where the triangular surrogate gradient changes sign, and which latency thresholds are valid. The tests record the chosen interpretation: strict greater-than firing follows fire() and the Leaky prose. The other boundary decisions are documented in the test specifications for review. The passing tests establish agreement with the selected interpretation; documentation consistency remains unresolved.

Pairwise coverage detects every required pair but does not cover every one of the 144 combinations or higher-order interactions. The reset model covers every abstract state and transition, but it does not exhaust numeric histories, concurrency or every possible sequence. Structural coverage covers seven selected library files; it records which Python lines and branches ran, not whether every behaviour is correct. The download tests use controlled doubles instead of live network services, and the only measured platform was Windows CPU.

3 3 Test results

In the early runs, I separated test-scaffolding faults from production defects. The first state tests copied model state incorrectly. The simulated download setup and a Windows-specific test substitute also needed repair. These were corrected and kept separate from library bugs.

Run progression

Run	Cases	Passed	Failed	Skipped
First LO3 run	1,121	90	1,031	0
After state-test setup repair, before library fixes	1,121	588	533	0
Final LO3 run	1,134	1,134	0	0
Existing tests in final run	68	68	0	0

The final LO3 run passed all 1,000 model cases and all nine rate-count checks. The README smoke test produced 25 time steps with 10 output units for one input. The training smoke test reported a loss of 0.25, finite gradients and changed weights.

Confirmed defects by area

Area	Confirmed defects	Finding summary
Synaptic state updates	1	A state-update error was detected by comparison with the independent Synaptic reference.
Surrogate gradients	4	Four backward-rule defects were exposed by formula-based gradient tests.
Reset behaviour	2	Two defects affected reset semantics or state isolation.
Archive and download security	3	Three defects affected archive extraction or secure download handling.
Total	10	Each production defect was counted once, even when several tests exposed it.

The mutation experiment introduced three deliberate faults in separate runs, and the tests detected all three. This shows that the selected assertions detect those seeded fault types. These three faults are the available mutation evidence; I do not report a package-wide mutation score.

3 4 Evaluation of results

I evaluated the suite using requirement traceability, input and boundary coverage, pairwise coverage, model state coverage, structural coverage, statistical acceptance and defect yield. The measures are summarized below; each describes a different part of test adequacy.

Measure	Result	Interpretation
Requirements with evidence	6/6 (100%)	Each requirement has a linked test or smoke artifact. R5 acceptance is still pending.
Selected-scope requirement pass	5/6	The selected automated scope passes for five requirements. Full sign-off is not claimed.
Input categories	135/137 (98.54%)	Two planned categories remain unexecuted: participant sessions and native existing-symbolic-link extraction.
Boundary cases	36/36 (100%)	Every boundary in the declared inventory was executed.
Feasible setting pairs	95/95 (100%)	All required pairs were covered by 12 selected cases from 144 possible combinations.
Model reset states and transitions	4/4 states; 16/16 transitions	All declared abstract states and actions were reached; all 1,000 distinct cases passed.
Executable lines	681/1,014 (67.16%)	Coverage is limited to the seven files named in the design scope.
Python branches	193/410 (47.07%)	More than half of measured Python branches remain unexecuted.

Measure	Result	Interpretation
Rate-count intervals	9/9 in range	All nine predeclared binomial checks passed for the selected probabilities and seeds.
Seeded fault detection	3/3	All three deliberately introduced faults were detected.

Defect yield

The suite found 10 confirmed defects across 1,134 distinct LO3 test items, or 0.88 defects per 100 test items. This ratio describes defect yield for this suite. Cases are not independent, and the 1,000 reset cases make up a large part of the denominator. A comparison with another suite would require the same counting rule and a similar mix of tests.

Confidence and remaining work

I interpret the independent numerical checks, exact reset comparisons, security canaries and successful seeded-fault experiments increase confidence in the behaviours exercised. The lower branch coverage, untested input categories, single operating environment and specification conflicts limit that confidence outside the selected scope. The evidence supports the conclusion that the suite is effective at finding faults in the tested dynamics, gradients, reset paths and archive/download cases.

My next steps are to:

- Run five representative usability sessions and apply the planned acceptance threshold of four successful completions within 30 minutes without instructor help.
- Run extraction against a real pre-existing symbolic link on a platform that permits link creation, and verify that an external canary remains unchanged.
- Expand tests in the measured gaps, especially unvisited branches, other neuron implementations, encoder options and dataset handling. Then run the declared supported Python, PyTorch and device combinations.
- Resolve the threshold, triangular-gradient and latency-threshold wording so future tests use one approved contract.
- Include the generated JUnit, environment, coverage, smoke and assessment files with the portfolio or in the repository so a reviewer can inspect the recorded run directly.

Supporting project sources

The repository links below point to the LO3 test branch. Generated run outputs are written under portfolio-testing/lo3/evidence/final by the documented runner [2].

[1] LO1 requirements analysis and LO2 test plan

[2] LO3 README and run instructions

[3] Functional and structural test code

[4] Model-state and security test code

[5] Design, pairwise and model-case manifests

[6] Assessment and mutation scripts

Run evidence referenced in this report: portfolio-testing/lo3/evidence/final/environment.json, portfolio.xml, upstream.xml, coverage.json, assessment.json, requirement-results.json and smoke.txt. The branch tree reviewed for this report contains the runner and test code; retain or submit these generated files alongside the report when the assessor needs the underlying records.